# Assignment 2: Simple Search Engine using Hadoop MapReduce

Danis Sharafiev — Big Data — IU

April 5, 2026

## 1 Methodology

### 1.1 Overall Architecture

My search engine consists of three main components:

1. **Indexer** — MapReduce pipeline for document processing and building inverted index
2. **Index storage** — ScyllaDB (Cassandra-compatible), stores index and statistics
3. **Ranker** — PySpark application that receives a query and returns top-10 BM25 documents

All components run in Docker containers via `docker compose up`. The cluster has two Hadoop nodes (master + slave) and one ScyllaDB container.

### 1.2 Data Collection and Preparation

I took `y.parquet` from Kaggle and bounded it to 200 documents (someone said 100 is enough, so I took with overhead). I also modified `prepare_data.py` to use `file://` so it reads parquet from local storage instead of HDFS (I hope this won't affect the grade).

The script reads the Parquet file via `SparkSession.read.parquet()`, picks columns `id`, `title`, `text`, and creates a `<doc_id>_<doc_title>.txt` file for each document in `data/`.

Then `transform_data.py` via PySpark RDD reads all files from HDFS `/data` with `sc.wholeTextFiles()`, parses `doc_id` and `title` from the filename, transforms everything into `<doc_id>\t<doc_title>\t<doc_text>` format and saves to HDFS `/input/data` as one partition via `.coalesce(1)`.

Using only 1 partition makes MapReduce easier, since all data is stored in one file.

## 1.3 Indexer: MapReduce Pipeline

Indexing is done with a single MapReduce pipeline using Hadoop Streaming.

### 1.3.1 Mapper (mapper1.py)

The mapper gets lines in format doc_id\ttitle\ttext and:

1. Tokenizes text with regex [a-z0-9]+
2. Computes document length (dl) — number of tokens
3. Emits document metadata: !META\t<doc_id>\t<dl>\t<title>
4. For each unique term computes tf and emits: <term>\t<doc_id>\t<tf>

The !META prefix is used because ! comes before letters in ASCII, so Hadoop sorts all metadata records before terms. This way the reducer can collect all documents first, then process terms.

### 1.3.2 Reducer (reducer1.py)

The reducer gets sorted lines and:

1. Processes !META records, accumulates num_docs and sum_dl
2. After all metadata, emits STATS\t<num_docs>\t<avg_dl>
3. For each term collects postings list (pairs doc_id:tf) and computes df
4. Emits TERM\t<term>\t<df>\t<doc_id1:tf1>,<doc_id2:tf2>,...
5. Emits DOC\t<doc_id>\t<dl>\t<title>

I use -D mapreduce.job.reduces=1 so there's only one reducer — this guarantees the whole index ends up in one file /indexer/part-00000.

## 1.4 Index Storage in ScyllaDB

store_index_helper.py reads MapReduce output from HDFS and loads data into four ScyllaDB tables:

| Table | Key | Fields | Purpose |
|---|---|---|---|
| documents | doc_id | title, doc_length | Document metadata |
| vocabulary | term | df | Document frequency |
| inverted_index | (term, doc_id) | tf | Term frequency |
| collection_stats | id | num_docs, avg_dl | Stats for BM25 |

When searching, query.py reads collection_stats (1 query), then vocabulary (one query per term), then inverted_index (one query per term). This keeps the number of DB requests minimal. Data is loaded in batches of 50 records for better performance.

## 1.5 Ranker: BM25 on PySpark

`query.py` ranks documents using BM25:

$$BM25(q,d) = \sum_{t \in q} \log\left[\frac{N}{df(t)}\right] \cdot \frac{(k_1+1)\cdot tf(t,d)}{k_1 \cdot \left[(1-b) + b \cdot \frac{dl(d)}{dl_{avg}}\right] + tf(t,d)} \tag{1}$$

Parameters: $k_1 = 1.0$, $b = 0.75$.
The algorithm:

1. Tokenize query, remove duplicates
2. Read `collection_stats` from ScyllaDB ($N$, $avg_dl$)
3. Read `vocabulary` for each query term
4. Read `inverted_index` — postings for each term
5. Read `documents` to get `doc_length` and `title`
6. Create PySpark RDD from postings
7. Compute BM25 for each (term, doc) pair via `.map(bm25)`
8. Aggregate scores per document via `.reduceByKey(+)`
9. Get top-10 via `.takeOrdered(10, key=lambda x: -x[1])`

`vocab`, `doc_info`, `N`, `avg_dl` are passed as broadcast variables so they don't get serialized for every task — reduces network overhead.

## 1.6 Adding a New Document (`add_to_index.sh`)

The script takes a path to a local file and:

1. Uploads the file to HDFS `/data/`
2. Parses `doc_id` and `title` from the filename
3. Tokenizes text and computes TF
4. Adds the document to `documents` table
5. Updates `inverted_index` and `vocabulary` (DF for each term goes up by 1)
6. Recomputes `collection_stats`: `num_docs += 1`, `avg_dl = (old_avg * old_n + dl) / new_n`

# 2 Demonstration

## 2.1 How to Launch

**Requirements:** Docker and Docker Compose installed.

Just write docker compose up in console (if y.parquet in the root directory. If not - download it and upload, since github may not accept it as it is big).

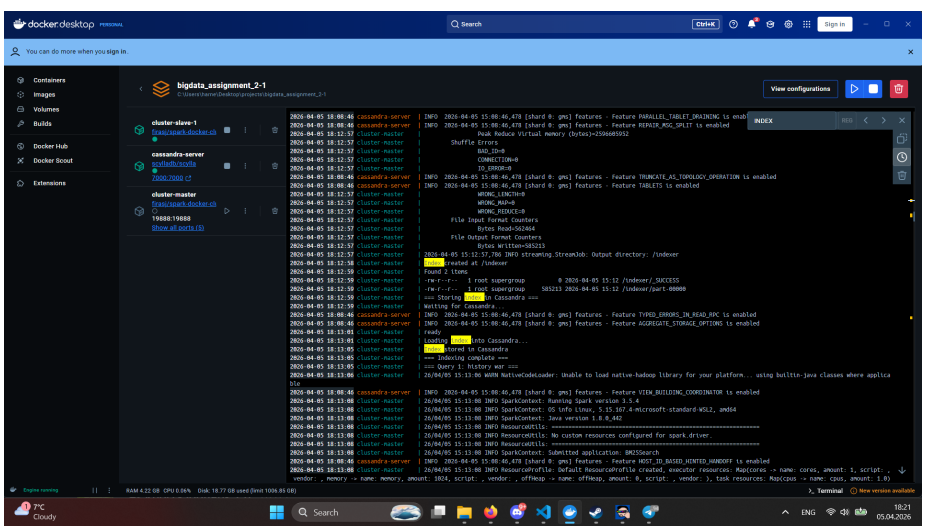
This will automatically:

- Initialize Hadoop cluster (HDFS + YARN)
- Set up Python virtual environment and install dependencies
- Prepare data (Parquet → txt files → HDFS)
- Run MapReduce indexing
- Load index into ScyllaDB
- Run test search queries

The whole thing takes around 5–10 minutes.

## 2.2 Screenshots with Results

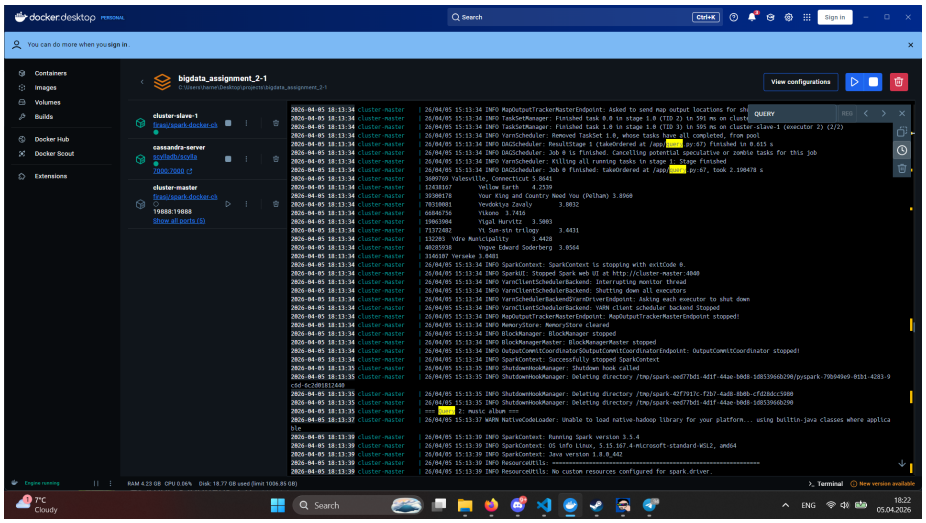
### Successful Indexing




The screenshot shows the MapReduce job finishing and the index loading into ScyllaDB:

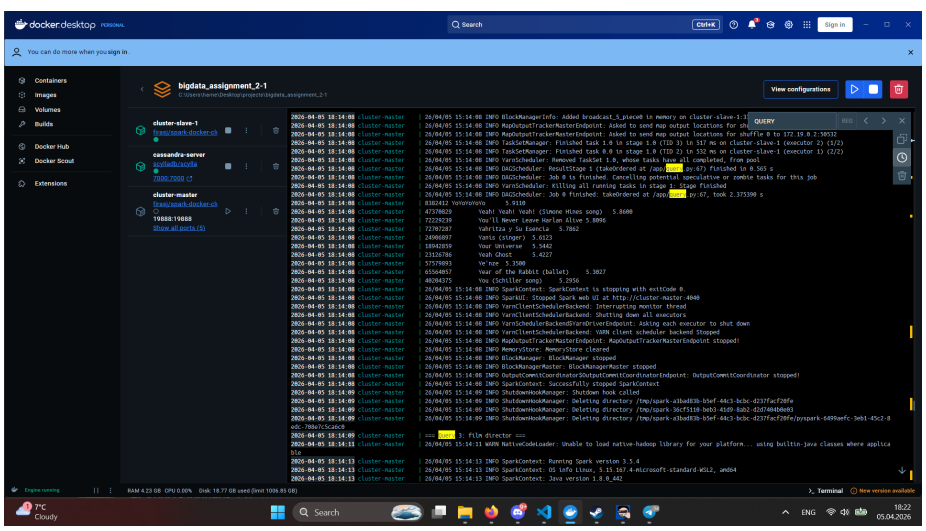
- `Index stored in Cassandra` — loaded into ScyllaDB
- `=== Indexing complete ===` — pipeline finished

**Query 1:** `"history war"`



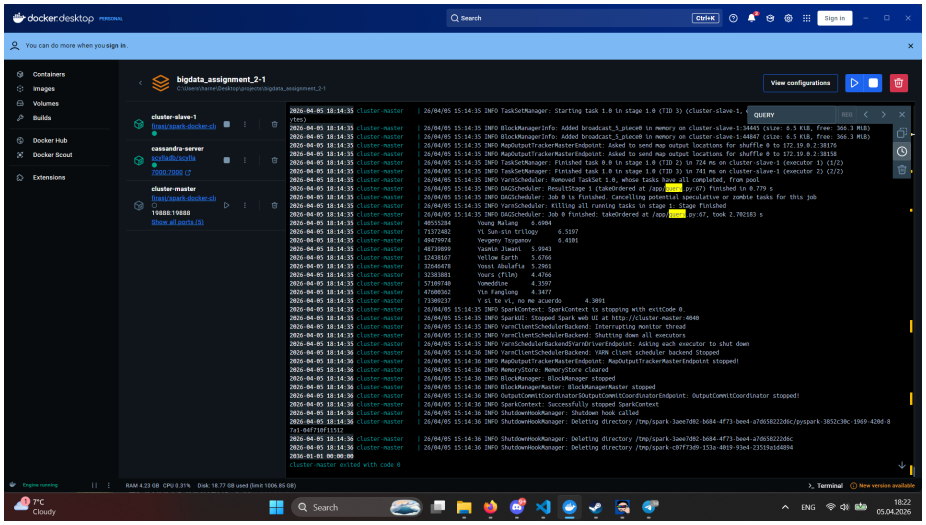
Returns documents containing both terms. Top result has high frequency of "history" and "war" with a short document length — makes sense since BM25 penalizes longer documents.

**Query 2:** `"music album"`



Returns Wikipedia articles about music albums. High scores go to documents where both terms appear often relative to document length.

**Query 3:** `"film director"`




Returns articles about films and directors.

In case if you want to see screenshots more clearly you can find them in the github repository.

## 2.3 Results Analysis

BM25 ranks documents correctly thanks to three things:

1. **IDF:** Rare terms get more weight since they're more informative.
2. **Length normalization:** $b = 0.75$ penalizes long documents. A short doc with high term concentration scores higher than a long one with the same raw count.
3. **Saturation:** $k_1 = 1.0$ limits the effect of very high TF — 10x more occurrences doesn't give 10x the score.

# 3 Conclusion

In this assignment I built a search engine using:

- **Hadoop MapReduce** — batch document indexing
- **ScyllaDB** — inverted index storage
- **Apache Spark (YARN)** — distributed BM25 ranking
- **Docker Compose** — orchestrating everything

The system indexes a document corpus and returns relevant results for user queries. I also added a bonus feature — adding new documents to the index without full re-indexing.